

Защитная речь — Smart Price (v3, под 15 слайдов, ~10 мин)

Темп: ~140 слов/мин. Текст ~1450 слов = **10 минут 10 секунд**.

Для каждого слайда: сколько он висит, что говорить дословно, какой акцент.

[в квадратных скобках курсивом] — можно опустить если поджимает время.

Слайд 1 · Титульный (0:00 — 0:30, 30 сек)

Уважаемые члены комиссии, уважаемый председатель.

Меня зовут **Санько Егор Александрович**, я студент группы **4-МД-16** Института информационных технологий и автоматизации, кафедры информационных и управляющих систем.

Представляю вашему вниманию выпускную квалификационную работу на тему «**Система метапоиска товаров на торговых площадках с функцией интеллектуального анализа цен**».

Научный руководитель — **Вагнер Виктория Игоревна**.

[Короткая пауза 1 сек, переключаешь слайд.]

Слайд 2 · Актуальность · Цель и задачи (0:30 — 2:00, 90 сек)

Левая половина — актуальность (45 сек)

За последние пять лет рынок электронной коммерции России и Беларуси вырос более чем в три раза. Параллельно с этим выросло и количество маркетплейсов — десятки крупных торговых площадок, у каждой свой ассортимент и своя цена.

Для покупателя это означает простую, но болезненную проблему: чтобы найти лучшую цену, ему нужно открыть **пять и более вкладок**, скопировать запрос, дождаться выдачи в каждой и сравнить результаты глазами. По нашим замерам, на это уходит **от восьми до пятнадцати минут**.

Ситуацию усугубляет динамическое ценообразование — разброс цены на один и тот же товар в один день достигает **двадцати — тридцати процентов**. Системных решений, объединяющих выдачу в едином пространстве России и Беларуси, на момент начала работы не существовало.

Правая половина — цель и задачи (45 сек)

Целью работы являлось проектирование и разработка веб-платформы AI-метапоиска товаров на маркетплейсах Беларуси и России.

Для достижения цели поставлены **пять задач**:

Первая — анализ предметной области и существующих агрегаторов.

Вторая — проектирование архитектуры с поддержкой потоковой передачи через Server-Sent Events.

Третья — разработка парсеров для шести маркетплейсов с обходом антибот-защит.

Четвёртая — интеграция AI-ассистента на базе **Gemini 2.5 Flash** через OpenRouter с поддержкой tool-use.

И пятая — тестирование и развёртывание системы в промышленную эксплуатацию.

[Акценты — «8–15 минут», «20–30 процентов разброс», «пять задач».]

Слайд 3 · Решение (2:00 — 2:45, 45 сек)

Ключевая идея решения — **параллельный опрос шести независимых источников и потоковая выдача результатов** пользователю по мере готовности каждого из них.

На схеме показан полный пайплайн: пользователь вводит запрос, backend через `asyncio.gather` стартует шесть парсеров одновременно, и каждый оффер сразу же отдаётся клиенту через Server-Sent Events. Клиент не ждёт самого медленного парсера — первая карточка появляется через **1,4 секунды**.

Параллельно работает **AI-слой** на Gemini 2.5 Flash — отвечает на вопрос «дорого это или дешево», опираясь на историю цен.

Слайд 4 · Демо · Поиск (2:45 — 3:15, 30 сек)

На слайде — реальный продакшен-скриншот. Пользователь ввёл просторечный запрос «**плойка 5**» — система через AI-коррекцию привела его к канонической форме «**PlayStation 5**», запустила параллельный поиск и нашла **38 предложений из пяти источников**. В шапке видны счётчики по каждому парсеру в реальном времени. Лучшая цена — Яндекс Маркет, тридцать девять тысяч сто семьдесят три рубля.

[Указать пальцем на стрелку «плойка 5 → PlayStation 5» — это эффектная демонстрация AI-коррекции.]

Слайд 5 · Архитектура (3:15 — 4:15, 60 сек)

Технически система построена на пяти слоях.

Клиент — Next.js 14 на TypeScript со статическим экспортом. Использует EventSource для приёма SSE-потока и Zustand для инкрементального обновления карточек.

Edge — nginx на VPS с поддержкой HTTP/2 и ALPN. Критично — отключено `proxy_buffering`, иначе nginx копит ответ и ломает стриминг.

Backend — FastAPI на Python 3.11, асинхронный SQLAlchemy 2.0. Ядро — `asyncio.gather` по шести парсерам с `yield` каждого оффера в SSE-поток.

Парсеры и источники — шесть отдельных async-модулей с единым контрактом `search(query) → list[Offer]`. *[Очередь задач — Celery поверх Redis, база — PostgreSQL 16.]*

Всё развёрнуто как шесть Docker-контейнеров на одном VPS-сервере.

Слайд 6 · Парсеры (4:15 — 4:50, 35 сек)

Из **двадцати пяти проанализированных маркетплейсов** отобраны **шесть** по критериям качества данных и стабильности доступа.

Это **Onliner.by** — единственный белорусский источник, JSON API. **Яндекс Маркет** — российский, HTML с регулярными выражениями. **Perapd, World Devices** — российский ритейл и импорт электроники. И **1click** с **BigGeek** — специализированный parallel-import техники Apple.

Архитектура парсеров модульная: новый источник подключается как отдельный async-модуль без изменения ядра.

Слайд 7 · AI-слой (4:50 — 5:25, 35 сек)

В качестве LLM выбран **Gemini 2.5 Flash от Google** через прокси-сервис **OpenRouter**.

Выбор обоснован тремя факторами: высокое качество понимания русского языка, потоковый API и стоимость — десять центов за миллион токенов, на порядок дешевле GPT-4.

Принцип работы — **паттерн tool-use**. Модель сама решает, какие инструменты бэкенда вызвать: историю цен, сравнение офферов, расчёт медианы. На вход — текущее предложение, на выход — структурированный вердикт «дорого, норма или дёшево» с обоснованием.

Слайд 8 · «Найти дешевле» — реверс WebSocket Алисы (5:25 — 6:25, 60 сек)

Это уникальная функция системы, **аналогов которой на рынке нет**.

Идея: товар прогоняется через **закрытый WebSocket-канал бэкенда Яндекс.Алисы** — тот самый, через который голосовой ассистент находит «то же самое дешевле». Публичного API у этого канала нет, поэтому в рамках работы выполнен **исследовательский reverse-engineering протокола**.

Алгоритм — три шага. Открываем WebSocket-сессию с UUID-эмуляцией клиента Яндекс. Парсим бинарный payload, извлекая типизированные сущности — `EAliceOfferCard`, `EAliceOffer`, `EAliceOfferList`. Мерджим найденные офферы в общую ленту по артикулу или EAN.

Справа на слайде — реальный скриншот: запрос **iPhone 17 Pro**, лучшая найденная цена **119 090 рублей**, что на **16 113 рублей дешевле**, чем на Ozon — то есть минус двенадцать процентов на конкретном кейсе. Фича работает в продакшене и даёт измеримый экономический эффект.

[Здесь нужна гордость в голосе — это центральный аргумент работы.]

Слайд 9 · Функционал (6:25 — 7:05, 40 сек)

За кулисами основного интерфейса работают **четыре подсистемы**.

AI-коррекция запросов приводит сленг и опечатки к канонической форме в среднем за четыреста двадцать миллисекунд с эффективностью девяносто семь случаев из ста.

Гибридная фильтрация — двухступенчатая. На первом этапе regex отсекает аксессуары, на втором AI-классификатор отделяет основной товар от сопутствующих. Релевантность выдачи выросла на **тридцать-сорок процентов**.

AI-ассистент с tool-use — все цифры берутся из реальных вызовов функций бэкенда, без галлюцинаций.

И **история цен** — динамика за шестьдесят дней по каждому товару.

Слайд 10 · Сравнение и анализ цен (7:05 — 7:30, 25 сек)

Кроме поиска, реализованы ещё две функции — слева **сравнение до четырёх товаров с AI-обоснованием**, справа — **анализ конкретной цены по 100-балльной шкале** относительно медианы рынка с автодетекцией региона BY или RU. На примере: iPhone 16 Pro за 79 480 рублей получает **95 баллов из 100** — отличное предложение.

Слайд 11 · Метрики (7:30 — 7:55, 25 сек)

Все цифры на слайде — **результат измерений на боевом VPS**, не оценочные расчёты. Время до первой карточки — **1,4 секунды**. Шесть источников параллельно. Ускорение по сравнению с ручным поиском — порядка ста раз. Объём кода — около пятнадцати тысяч строк на TypeScript и Python. Экономия через функцию Алисы — десять-сорок процентов.

Слайд 12 · Экономика (7:55 — 8:35, 40 сек)

Эксплуатация системы рентабельна. **Стек расходов**: VPS — пятнадцать долларов в месяц, OpenRouter — пять, домен и SSL — два. Итого **двадцать два доллара в месяц** за полностью работающий сервис.

При средней ставке IT-специалиста пятнадцать долларов в час экономия времени пользователя оценивается в два с половиной — семь с половиной долларов за один поиск. То есть **трёх-девяти использований достаточно, чтобы окупить весь месяц работы сервиса**.

Возможные модели монетизации — контекстная реклама маркетплейсов, партнёрская программа с CPA-выплатами или премиум-подписка с расширенной аналитикой.

Слайд 13 · Инженерные вызовы (8:35 — 9:30, 55 сек)

Четыре основных инженерных вызова, ради которых работа и существовала.

Первое — гетерогенные антибот-защиты шести маркетплейсов. Один универсальный прокси не работает: для каждого источника свой стек. Onliner — открытый JSON API, Яндекс.Маркет — headless-браузер с прокруткой страницы для загрузки SKU, Репард, 1click и BigGeek — HTTP-клиент с ротацией User-Agent и timing-jitter, World Devices — обход кэша миниатюр. В итоге — **шесть парсеров стабильно на проде, около девяноста пяти процентов успеха** на типовых запросах.

Второе — потоковый pipeline. Параллельный опрос через `asyncio.gather` плюс потоковая отдача через `EventSourceResponse`. Главная грабля — nginx по умолчанию буферизует ответ и ломает SSE. Конфигурация `proxy_buffering off`, `gzip off` на стримовых роутах, HTTP/2 ALPN. **Первая карточка через 1,4 секунды.**

Третье — реверс бинарного WebSocket-протокола Яндекс.Алисы, о чём я уже говорил.

Четвёртое — intent-aware фильтрация. Был баг: поиск «чехол iPhone 16 Pro» возвращал четыре результата вместо двадцати пяти, потому что парсеры безусловно отсеивали аксессуары. Решение — детерминистический intent-детектор по семидесяти пяти regex-паттернам до обращения к LLM. **Результат на проде: четыре стало двадцать пять, Яндекс.Маркет с нуля поднялся до двадцати.**

[Если поджимает время — четвёртый пункт можно опустить, но именно он показывает свежую инженерную итерацию.]

Слайд 14 · Заключение (9:30 — 10:00, 30 сек)

Подведу итоги.

Цель работы достигнута. Спроектирована и реализована веб-платформа AI-метапоиска цен, развёрнута на боевом VPS, доступна по адресу **smrt-price.ru** и в настоящий момент обрабатывает реальные пользовательские запросы.

Все **пять задач** работы решены — от анализа предметной области до развёртывания в продакшен. Все **ключевые архитектурные и AI-решения подтверждены измерениями на проде**: TTFR 1,4 секунды, шесть источников, ускорение в сто раз, типовая экономия пользователя десять-сорок процентов при стоимости эксплуатации двадцать два доллара в месяц.

Слайд 15 · Финал (10:00 — 10:10, 10 сек)

Доклад окончен. Спасибо за внимание. Готов ответить на ваши вопросы.

Тайм-чарт (шпаргалка)

№	Слайд	Тайм-код	Длит.	Ключевая мысль
1	Титул	0:00	0:30	Приветствие, тема, ФИО, руководитель
2	Актуальность · Цель · Задачи	0:30	1:30	8–15 мин ручного → 5 задач
3	Решение	2:00	0:45	6 источников + AI-вердикт
4	Демо · Поиск	2:45	0:30	«плойка 5 → PlayStation 5»
5	Архитектура	3:15	1:00	5 слоёв, SSE + asyncio.gather
6	Парсеры	4:15	0:35	25 → 6, модульно
7	AI-слой	4:50	0:35	Gemini 2.5 Flash, tool-use
8	Killer feature	5:25	1:00	iPhone 17 Pro: –16 113 ₽ vs Ozon
9	Функционал	6:25	0:40	4 подсистемы за кулисами
10	Сравнение/Анализ	7:05	0:25	95/100 баллов
11	Метрики	7:30	0:25	1,4 с · ~100× · ~15K
12	Экономика	7:55	0:40	\$22/мес, окупаемость 3–9 использований
13	Инженерные вызовы	8:35	0:55	4 вызова, intent-фильтр 4 → 25
14	Заключение	9:30	0:30	Цель достигнута, всё измерено
15	Финал	10:00	0:10	Спасибо, вопросы
Итого			~10:10	

Шпаргалка цифр

Метрика	Значение
TTFR (отклик)	1,4 с
Первый батч	5,8 с
Полный поиск	24,2 с
Источников активно	6
Изучено маркетплейсов	25
Ускорение vs ручной	~100x
Стоимость эксплуатации	~\$22/мес
Экономия пользователю	\$2,5–7,5 за поиск
Окупаемость	3–9 использований
AI-коррекция, время	420 мс
AI-коррекция, точность	97 / 100
Гибридная фильтрация, +релевантности	+30–40%
Антигаллюцинационные тесты	9 / 10
Экономия через Алису	10–40%
iPhone 17 Pro vs Ozon	–16 113 Р (–12%)
Intent-фильтр «чехол iPhone 16 Pro»	4 → 25 результатов
Строк кода	~15 000

Заготовки ответов на вопросы

«Почему SSE, а не WebSocket?»

Серверу нужна только односторонняя передача — поток событий «нашёл, нашёл, нашёл». SSE проще, держит автореконнект из коробки, прозрачно проксируется через nginx и не требует дополнительной библиотеки на фронте. WebSocket был бы избыточен.

«Реверс WebSocket Алисы — это законно?»

Это исследовательский reverse-engineering в рамках академической работы. Условия использования не нарушены — мы работаем с трафиком собственного браузерного клиента, который любой пользователь Яндекс может наблюдать у себя в DevTools.

«А если Яндекс изменит протокол?»

Признаваемый риск. Функция «Найти дешевле» вынесена в отдельный воркер, изолирована от ядра. Если протокол меняется — отключается только эта фича, основной метапоиск по шести источникам продолжает работать. Предусмотрен мониторинг успешности WS-сессии.

«Почему Gemini, а не GPT-4 или Claude?»

Gemini 2.5 Flash через OpenRouter дешевле GPT-4o Mini в полтора раза и Claude Haiku в два с половиной раза, при сопоставимом качестве для tool-use и нормализации текста. Кроме того, OpenRouter решает проблему геоблокировки прямого Gemini API из России. Через OpenRouter можно горячо переключиться на любую другую модель без изменения кода.

«А что с Wildberries и Ozon — почему их нет?»

Wildberries был временно отключён из-за нестабильности полей наличия и попадания в выдачу восстановленных товаров вместе с новыми. Ozon — изначально не подключали из-за агрессивной антибот-защиты, требующей отдельной инфраструктуры. Архитектура такова, что новый источник подключается как отдельный модуль — ядро системы менять не нужно.

«Что такое intent-aware фильтр? Это вы добавили недавно?»

Это последняя инженерная итерация перед защитой. Раньше система не различала запрос «iPhone» от запроса «чехол на iPhone» — для парсеров оба запроса означали «отсей всё, что выглядит как аксессуар». Я добавил детерминистический детектор намерения по семидесяти пяти regex-паттернам, который срабатывает до обращения к LLM. Это даёт стопроцентно воспроизводимое поведение и фикс реального продакшен-бага.

«Какая средняя загрузка вашего VPS?»

На пиках до пятидесяти процентов CPU при пяти-десяти одновременных поисках. Бутылочное горлышко — не процессор, а сами маркетплейсы: ответы парсеров приходят за 0,9–1,8 секунды. По памяти — занимаем около 1,5 ГБ из 4 ГБ доступных.

«Как тестировали 97% точности AI-коррекции?»

Сто типовых пользовательских запросов с намеренными опечатками, сленгом, транслитерацией — «айфон 16 пра», «сяоми 15», «плойка 5». Результат вручную сверялся с эталонной канонической формой. В 97 из 100 случаев модель привела запрос корректно. Подробная таблица — в разделе 4.2.2 пояснительной записки.

«Будет ли мобильное приложение?»

Архитектура изначально модульная — фронтенд общается с бэкендом по REST/SSE. Мобильное приложение на Expo/React Native планируется как отдельная итерация и подключается к тому же API без изменений на стороне сервера.

Финальные напоминания

- **Главные акценты:** слайды 2 («8–15 минут»), 8 («–16 113 Р vs Ozon»), 11 («измерено в проде»), 13 («4 → 25»).
- **Если поджимает время** — пропускай абзацы *[в курсиве]*, особенно про intent-фильтр на слайде 13. Это сэкономит ~20 сек.
- **Если идёшь с запасом** — добавь на слайде 12 строчку про «модели монетизации», на слайде 14 — упомяни 100 страниц / 23 таблицы.
- **Темп:** ~140 слов в минуту. Не торопись закончить — лучше пауза, чем скороговорка.
- **На «спасибо за внимание»** — короткая пауза, кивок, тут же садись.